Sessie I: Haskell de basics

PAL-sessie Declaratieve Talen

Yarne Ramakers, Yendric Van Roey

26 oktober 2026

KU Leuven

Zijn er vragen vanuit de oefenzittingen?

Planning voor vandaag

1. **Nuttige functies uit de standard library**
   - even opfrissen: `.`, `$` en `flip`
   - `curry` en `uncurry`
   - `swap`
   - `zip`, `unzip` en `zipWith`
   - `span` en `break`
   - `sort` en `sortOn`
   - documentatie zoeken en `hlint`
2. **Lazy evaluation**
3. **Hlint**
4. **Opdrachten**

Nuttige functies uit de Prelude

Even opfrissen: `.`, `$` en `flip`

```
(.)  :: (b → c) → (a → b) → a → c        (f . g) x  =  f (g x)
($)  :: (a → b) → a → b                  f $ x      =  f x
flip :: (a → b → c) → b → a → c          flip f x y =  f y x
```

`$` heeft de laagste prioriteit en bespaart je zo haakjes. Drie keer hetzelfde:

```
length (filter (> 3) (map (* 2) xs))
length $ filter (> 3) $ map (* 2) xs
(length . filter (> 3) . map (* 2)) xs
```

`flip` wisselt de eerste twee argumenten om:

```
flip replicate 'a' 3 == "aaa"
```

Kleine quiz

Gegeven:

```
zipWith :: (a → b → c) → [a] → [b] → [c]
flip    :: (x → y → z) → y → x → z
```

Wat is het type van `flip zipWith`?

A `(b → a → c) → [a] → [b] → [c]`

B `(a → b → c) → [b] → [a] → [c]`

C `[a] → (a → b → c) → [b] → [c]`

D `[c] → [b] → [a] → (a → b → c)`

Kleine quiz: antwoord C

Elke functie neemt maar één argument, dus lees `zipWith` als:

```
zipWith :: (a → b → c) → ([a] → ([b] → [c]))
           -------------    ---    ------------
                 x           y          z
```

`flip` wisselt enkel de eerste twee argumenten om; de rest (`z`) blijft staan:

```
flip zipWith :: [a] → (a → b → c) → [b] → [c]

flip zipWith [1, 2, 3] (+) [10, 20, 30]  == [11, 22, 33]
```

Kleine quiz: antwoord C

Elke functie neemt maar één argument, dus lees `zipWith` als:

```
zipWith :: (a → b → c) → ([a] → ([b] → [c]))
           -------------    ---    ------------
                 x           y          z
```

`flip` wisselt enkel de eerste twee argumenten om; de rest (`z`) blijft staan:

```
flip zipWith :: [a] → (a → b → c) → [b] → [c]

flip zipWith [1, 2, 3] (+) [10, 20, 30]  == [11, 22, 33]
```

A wisselt de argumenten van de functie binnenin: dat doet `flip` niet
B wisselt de laatste twee argumenten, maar `flip` pakt de eerste twee
D draait het hele type om; het resultaat blijft `[c]`

Elke functie neemt maar één argument

De pijl → is rechts-associatief. Deze twee types zijn dus hetzelfde

```
replicate :: Int → a → [a]
replicate :: Int → (a → [a])
```

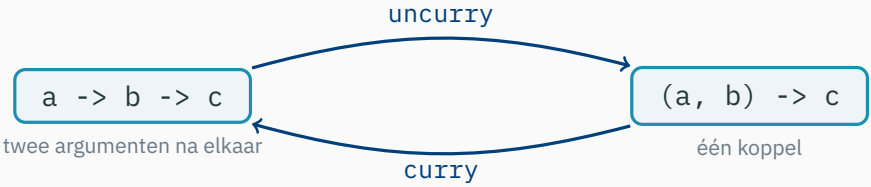
Je kan ook maar een deel van de argumenten geven (partiële toepassing):

```
replicate 3      :: a → [a]          -- een nieuwe functie!
replicate 3 'x'  :: [Char]           -- "xxx"
```

Soms zitten de argumenten echter samen in een koppel:

```
optellen :: (Int, Int) → Int        -- maar één argument: een tuple
optellen (x, y) = x + y             -- hoe zet je de ene vorm om in de andere?
```

curry en uncurry




```
uncurry :: (a → b → c) → ((a, b) → c)        uncurry f (x, y) = f x y
curry   :: ((a, b) → c) → (a → b → c)        curry f x y      = f (x, y)
```

Bijvoorbeeld:

```
uncurry replicate :: (Int, a) → [a]
curry optellen    :: Int → Int → Int
```

Kleine quiz: `curry` en `uncurry`

Wat is het resultaat?

```
uncurry replicate (3, 'x')                          -- ?
curry fst 'a' True                                  -- ?
map (uncurry (++)) [("ab", "c"), ("", "de")]        -- ?
```

En wat is het type van `uncurry (++)`?

Kleine quiz: `curry` en `uncurry`

Wat is het resultaat?

```
uncurry replicate (3, 'x')                    -- "xxx"
curry fst 'a' True                            -- 'a'
map (uncurry (++)) [("ab", "c"), ("", "de")]  -- ["abc", "de"]
```

En wat is het type van `uncurry (++)`?

Kleine quiz: `curry` en `uncurry`

Wat is het resultaat?

```
uncurry replicate (3, 'x')                    -- "xxx"
curry fst 'a' True                            -- 'a'
map (uncurry (++)) [("ab", "c"), ("", "de")]  -- ["abc", "de"]
```

En wat is het type van `uncurry (++)`?

```
uncurry (++) :: ([a], [a]) → [a]
```

Swap en tuples

`swap` wisselt de twee elementen van een koppel om.

Zit niet in de Prelude! `import Data.Tuple (swap)`

```haskell
swap :: (a, b) → (b, a)
swap (x, y) = (y, x)
```

```haskell
swap (1, "een") == ("een", 1)
map swap [(1, 'a'), (2, 'b')] == [('a', 1), ('b', 2)]
```

Swap en tuples

`swap` wisselt de twee elementen van een koppel om.
Zit niet in de Prelude! **`import`** `Data.Tuple (swap)`

```
swap :: (a, b) → (b, a)
swap (x, y) = (y, x)
```

```
swap (1, "een") == ("een", 1)
map swap [(1, 'a'), (2, 'b')] == [('a', 1), ('b', 2)]
```

`swap` werkt op een koppel, `flip` op de argumenten van een functie. Met `curry` en `uncurry` zie je het verband:

```
curry swap :: a → b → (b, a)          -- curry swap 1 'x' == ('x', 1)
uncurry (flip replicate) ('z', 3)  ==  uncurry replicate (swap ('z', 3))
```

`flip` en `swap`: twee kanten van hetzelfde idee

Met `curry` en `uncurry` kan je de ene uitdrukken in de andere:

```
flip' :: (a → b → c) → b → a → c


swap' :: (a, b) → (b, a)
```

`flip` en `swap`: twee kanten van hetzelfde idee

Met `curry` en `uncurry` kan je de ene uitdrukken in de andere:

```
flip' :: (a → b → c) → b → a → c
flip' f = curry (uncurry f . swap)

swap' :: (a, b) → (b, a)
```

`flip` en `swap`: twee kanten van hetzelfde idee

Met `curry` en `uncurry` kan je de ene uitdrukken in de andere:

```haskell
flip' :: (a → b → c) → b → a → c
flip' f = curry (uncurry f . swap)

swap' :: (a, b) → (b, a)
swap' = uncurry (flip (,))          -- (,) :: a → b → (a, b)  =  curry id
```

`flip` en `swap`: twee kanten van hetzelfde idee

Met `curry` en `uncurry` kan je de ene uitdrukken in de andere:

```
flip' :: (a → b → c) → b → a → c
flip' f = curry (uncurry f . swap)

swap' :: (a, b) → (b, a)
swap' = uncurry (flip (,))          -- (,) :: a → b → (a, b)  =  curry id
```

```
  flip' f x y
= curry (uncurry f . swap) x y
= (uncurry f . swap) (x, y)
= uncurry f (swap (x, y))
= uncurry f (y, x)
= f y x
```

```
  swap' (x, y)
= uncurry (flip (,)) (x, y)
= flip (,) x y
= (,) y x
= (y, x)
```

zip

`zip :: [a] → [b] → [(a, b)]` koppelt twee lijsten element per element.
Het stopt bij de kortste lijst:

```
zip :: [a] → [b] → [(a, b)]
zip (x : xs) (y : ys) = (x, y) : zip xs ys
zip _        _        = []

zip [1, 2, 4] ['a', 'b', 'd']  == [(1, 'a'), (2, 'b'), (4, 'd')]
zip [1, 2, 3] "ab"             == [(1, 'a'), (2, 'b')]
```

Daardoor mag één van de lijsten zelfs oneindig zijn:

```
enumerate :: [a] → [(Int, a)]
enumerate xs = zip [0 ..] xs

enumerate "Hoi" == [(0, 'H'), (1, 'o'), (2, 'i')]
```

unzip en zipWith

`unzip :: [(a, b)] → ([a], [b])` doet precies het omgekeerde van `zip`:

```
unzip [(1, 'I'), (5, 'V'), (10, 'X')] == ([1, 5, 10], "IVX")
```

Met `zipWith` kies je zelf wat je met de twee elementen doet:

```
zipWith :: (a → b → c) → [a] → [b] → [c]
zipWith (+)       [1, 2, 3] [10, 20, 30]  == [11, 22, 33]
zipWith replicate [1, 2, 3] "abc"         == ["a", "bb", "ccc"]
```

`zip` is dus een speciaal geval van `zipWith`:

```
zip xs ys == zipWith (,) xs ys
```

span en break

Beide hebben hetzelfde type: `(a → Bool) → [a] → ([a], [a])`

Werking:

```
span p xs = (takeWhile p xs, dropWhile p xs) -- zolang p waar is in eerste,
                                             -- alles erachter in tweede
break p = span (not . p)                     -- tegengestelde van span
```

Voorbeeld (lijst splitsen):

```
span (<= 3) [1..6] == ([1, 2, 3], [4, 5, 6])
span (<= 3) [1, 2, 3, 4, 3] == ([1, 2, 3], [4, 3]) -- tweede drie komt rechts
break (== ' ') "hello world yendric"  ==  ("hello", " world yendric")
```

sort en sortOn

Dienen geïmporteerd te worden:

```
import Data.List (sort, sortOn)
```

Werking:

```
sort :: Ord a ⇒ [a] → [a]                -- sorteert een lijst
sortOn :: Ord b ⇒ (a → b) → [a] → [a] -- sorteer originele waardes op basis van
                                  -- gemapte waardes
```

Voorbeeld:

```
sort [('C', 0), ('A', 2), ('B', 1)] == [('A', 2), ('B', 1), ('C', 0)]
sortOn snd [('C', 0), ('A', 2), ('B', 1)] == [('C', 0), ('B', 1), ('A', 2)]
```

Lazy evaluation

Lazy evaluation: pas rekenen als het moet

Haskell rekent een expressie pas uit wanneer het resultaat echt nodig is. Tot dan is het een thunk: een uitgestelde berekening. `undefined` crasht zodra je hem uitrekent, maar hier gebeurt dat nooit:

```
fst (1, undefined)  == 1        length [undefined, undefined]  == 2
```

Lazy evaluation: pas rekenen als het moet

Haskell rekent een expressie pas uit wanneer het resultaat echt nodig is. Tot dan is het een thunk: een uitgestelde berekening. `undefined` crasht zodra je hem uitrekent, maar hier gebeurt dat nooit:

```
fst (1, undefined)  == 1        length [undefined, undefined]  == 2
```

Wat gebeurt er bij `head (map (*2) [1..])`?

```
  head (map (* 2) [1 ..])
= head (map (* 2) (1 : [2 ..]))
= head (1 * 2 : map (* 2) [2 ..])     -- de staart blijft een thunk
= 1 * 2
= 2                                   -- maar één element uitgerekend!
```

Thunks bekijken in GHCi

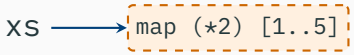
Met `:sprint` toont GHCi een waarde zonder ze uit te rekenen. Een `_` is een thunk.

```
ghci> xs = map (*2) [1..5] :: [Int]
ghci> :sprint xs
xs = _
ghci> head xs
2
ghci> :sprint xs
xs = 2 : _
ghci> length xs
5
ghci> :sprint xs
xs = [2,_,_,_,_]
```

- `head` heeft enkel het eerste element nodig
- `length` loopt de hele structuur af, maar rekent de elementen zelf niet uit
- pas na `sum xs` staat er `[2,4,6,8,10]`
- eens uitgerekend, blijft een waarde uitgerekend

Vergeet `:: [Int]` niet, anders toont `:sprint` altijd `_`.

Een thunk in het geheugen

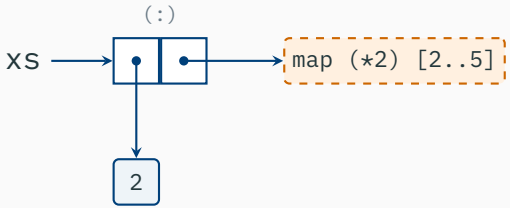


`1*2` thunk `2` uitgerekende waarde `(:)`-cel: element en rest

`xs = map (*2) [1..5]`: er is nog niets uitgerekend, enkel een recept.

`:sprint xs` → `_`

Een thunk in het geheugen

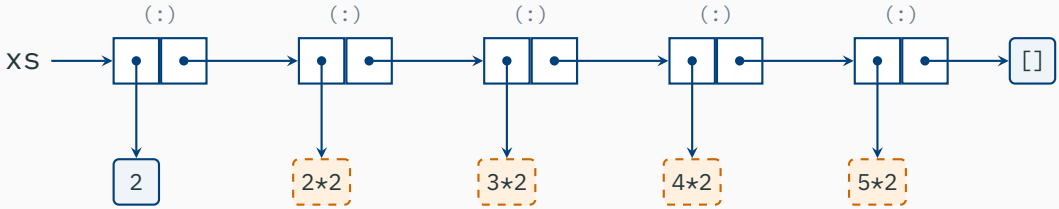


`1*2` thunk `2` uitgerekende waarde `(:)`-cel: element en rest

Na `head xs`: één `(:)`-cel en één waarde. De rest blijft een thunk.

```
:sprint xs  →  2 : _
```

Een thunk in het geheugen

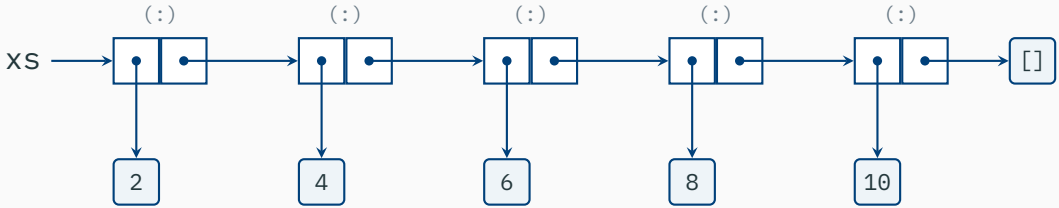


1*2 thunk 2 uitgerekende waarde (:)-cel: element en rest

Na `length xs`: de hele structuur bestaat, de elementen zijn nog thunks.

`:sprint xs` → `[2,_,_,_,_]`

Een thunk in het geheugen



`1*2` thunk `2` uitgerekende waarde `(:)`-cel: element en rest

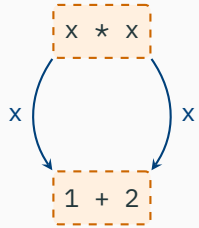
Na `sum xs`: elke thunk werd overschreven met zijn waarde.

`:sprint xs` → `[2,4,6,8,10]`

Sharing: een thunk wordt maar één keer uitgerekend

```
kwadraat :: Int → Int
kwadraat x = x * x

kwadraat (1 + 2)
```

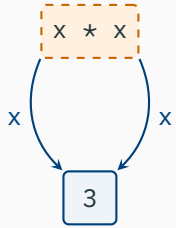


Haskell plakt `1 + 2` niet twee keer in de body. Beide `x`'en wijzen naar dezelfde thunk in het geheugen.

Sharing: een thunk wordt maar één keer uitgerekend

```
kwadraat :: Int → Int
kwadraat x = x * x

kwadraat (1 + 2)
```



De eerste x die nodig is, rekent de thunk uit en overschrijft hem met 3. De tweede x ziet dus meteen 3.

Sharing: een thunk wordt maar één keer uitgerekend

```
kwadraat :: Int → Int
kwadraat x = x * x

kwadraat (1 + 2)
```

9

3

Resultaat: 9, en 1 + 2 werd maar één keer uitgerekend. Niemand wijst nog naar 3, dus dat ruimt de garbage collector op.

Oneindige lijsten

```
[1 ..]              -- 1, 2, 3, 4, ...
repeat 'x'          -- 'x', 'x', 'x', ...
iterate (* 2) 1     -- 1, 2, 4, 8, 16, ...
cycle [1, 2, 3]     -- 1, 2, 3, 1, 2, 3, ...
```

Geen probleem, zolang je maar een eindig stuk ervan gebruikt:

```
take 6 (iterate (* 2) 1)             == [1, 2, 4, 8, 16, 32]
takeWhile (< 50) (map (^ 2) [1 ..])  == [1, 4, 9, 16, 25, 36, 49]
zip [0 ..] "Hoi"                     == [(0, 'H'), (1, 'o'), (2, 'i')]
```

Oneindige lijsten

```
[1 ..]              -- 1, 2, 3, 4, ...
repeat 'x'          -- 'x', 'x', 'x', ...
iterate (* 2) 1     -- 1, 2, 4, 8, 16, ...
cycle [1, 2, 3]     -- 1, 2, 3, 1, 2, 3, ...
```

Geen probleem, zolang je maar een eindig stuk ervan gebruikt:

```
take 6 (iterate (* 2) 1)             == [1, 2, 4, 8, 16, 32]
takeWhile (< 50) (map (^ 2) [1 ..])  == [1, 4, 9, 16, 25, 36, 49]
zip [0 ..] "Hoi"                     == [(0, 'H'), (1, 'o'), (2, 'i')]
```

Waarom nuttig? Je scheidt wat je berekent van hoeveel je ervan nodig hebt: “alle kwadraten” beschrijf je één keer, wie ze gebruikt beslist waar te stoppen.

Valkuilen

Alles wat de hele lijst nodig heeft, stopt nooit:

```
length [1 ..]        -- blijft eeuwig rekenen (onderbreek met Ctrl+C)
sum [1 ..]           -- idem, net als last, reverse, ...
```

Valkuilen

Alles wat de hele lijst nodig heeft, stopt nooit:

```
length [1 ..]       -- blijft eeuwig rekenen (onderbreek met Ctrl+C)
sum [1 ..]          -- idem, net als last, reverse, ...
```

`filter` versus `takeWhile`

```
takeWhile (< 10) [1 ..]   == [1, 2, 3, 4, 5, 6, 7, 8, 9]
filter    (< 10) [1 ..]   -- [1,2,3,4,5,6,7,8,9   en dan... niets meer
```

Waarom?

`takeWhile` stopt bij het eerste element dat niet voldoet. `filter` weet niet dat er geen kleinere getallen meer komen, en blijft zoeken.

HLint: laat je code nakijken

HLint leest je code en stelt eenvoudigere, idiomatischere alternatieven voor.

Typische beginnerscode:

```
allPositive :: [Int] → Bool
allPositive xs = if and (map (\x → x > 0) xs) == True then True else False
```

HLint: laat je code nakijken

HLint leest je code en stelt eenvoudigere, idiomatischere alternatieven voor.

Typische beginnerscode:

```
allPositive :: [Int] → Bool
allPositive xs = if and (map (\x → x > 0) xs) == True then True else False
```

```
$ hlint Voorbeeld.hs
Voorbeeld.hs:2:18-75: Warning: Redundant if
Voorbeeld.hs:2:21-46: Warning: Use all
Voorbeeld.hs:2:21-54: Warning: Redundant ==
Voorbeeld.hs:2:30-42: Suggestion: Avoid lambda using `infix'
4 hints
```

HLint: hints toepassen

Pas alle sugesties toe en je krijgt:

```
allPositive :: [Int] → Bool
allPositive xs = all (> 0) xs
```

```
$ hlint Voorbeeld.hs
Voorbeeld.hs:2:18-75: Warning: Eta reduce
1 hints
```

Pas toe en je krijgt:

```
allPositive :: [Int] → Bool
allPositive = all (> 0)
```

Vragen?

Opdracht: Advent of Code

Hoe denkt een functionele programmeur?

1. **Wat gaat erin, wat komt eruit?**
   Schrijf eerst het type van het hele probleem, bv. `solve :: String → Int`.
2. **Splits op in kleinere stappen**
   Schrijf enkel de signatures, met `= undefined` als body. Dat compileert al!
3. **Vul elk stukje apart in**
   Elke functie is klein en apart te testen in GHCi.
4. **Stel samen**
   De oplossing is een pipeline van kleine functies, aan elkaar gehangen met `.`

Hoe denkt een functionele programmeur?

1. **Wat gaat erin, wat komt eruit?**
   Schrijf eerst het type van het hele probleem, bv. `solve :: String → Int`.
2. **Splits op in kleinere stappen**
   Schrijf enkel de signatures, met `= undefined` als body. Dat compileert al!
3. **Vul elk stukje apart in**
   Elke functie is klein en apart te testen in GHCi.
4. **Stel samen**
   De oplossing is een pipeline van kleine functies, aan elkaar gehangen met `.`

Waarom types eerst?

Een type is een belofte: zegt wat een functie doet, voor je weet hoe. Vaak vertelt het type je zelfs al hoe de implementatie eruit moet zien.

Invoer verwerken

Advent of Code geeft je altijd één grote `String`.

```
lines "3   4\n1   7"      -- ["3   4", "1   7"]
words "3   4"             -- ["3", "4"]
read "42" :: Int          -- 42
```

Invoer verwerken

Advent of Code geeft je altijd één grote `String`.

```
lines "3   4\n1   7"      -- ["3   4", "1   7"]
words "3   4"             -- ["3", "4"]
read "42" :: Int          -- 42
```

`read` moet weten welk type het moet maken. Dat haalt Haskell uit je typesignatuur, of je schrijft het er zelf bij met `:: Int`.

Invoer verwerken

Advent of Code geeft je altijd één grote `String`.

```
lines "3   4\n1   7"      -- ["3   4", "1   7"]
words "3   4"             -- ["3", "4"]
read "42" :: Int          -- 42
```

`read` moet weten welk type het moet maken. Dat haalt Haskell uit je typesignatuur, of je schrijft het er zelf bij met `:: Int`.

Het patroon dat je bijna elke dag nodig hebt:

```
parse :: String → [[Int]]
parse = map (map read . words) . lines
-- "1 2 3\n4 5 6"  →  [[1,2,3],[4,5,6]]
```

Bestanden uitlezen

Via IO monad

```
main :: IO ()
main = do
  input ← readFile "input.txt"
  solve input
```

Historian Hysteria (AoC 2024, dag 1)

De invoer bestaat uit twee kolommen getallen.

- koppel het kleinste getal links aan het kleinste getal rechts, het op één na kleinste aan het op één na kleinste, ...
- tel de afstanden tussen alle koppels op

`adventofcode.com/2024/day/1`

Invoer:

```
3   4
1   7
5   2
```

Historian Hysteria (AoC 2024, dag 1)

De invoer bestaat uit twee kolommen getallen.

- koppel het kleinste getal links aan het kleinste getal rechts, het op één na kleinste aan het op één na kleinste, ...
- tel de afstanden tussen alle koppels op

`adventofcode.com/2024/day/1`

Invoer:

```
3   4
1   7
5   2
```

gesorteerd: `[1,3,5]` en `[2,4,7]`
afstanden: $1 + 1 + 2$

Uitvoer: **4**

Opdracht 1: begin bij de types

Stap 1: wat gaat erin, wat komt eruit?

```
solve :: String → Int
```

Opdracht 1: begin bij de types

Stap 1: wat gaat erin, wat komt eruit?

```
solve :: String → Int
```

Stap 2: in welke kleinere stukken kan je dat opsplitsen?

```
parse         :: String → ([Int], [Int])
totalDistance :: [Int] → [Int] → Int
distance      :: Int → Int → Int
```

Opdracht 1: begin bij de types

Stap 1: wat gaat erin, wat komt eruit?

```
solve :: String → Int
```

Stap 2: in welke kleinere stukken kan je dat opsplitsen?

```
parse         :: String → ([Int], [Int])
totalDistance :: [Int] → [Int] → Int
distance      :: Int → Int → Int
```

Stap 3: zet overal `= undefined`, vul de functies één voor één in en test elk stuk apart in GHCi. Pas op het einde stel je `solve` samen.

Oplossing 1: invoer parsen

```
parse :: String → ([Int], [Int])
parse = unzip . map (toPair . map read . words) . lines
  where toPair [x, y] = (x, y)
```

Oplossing 1: invoer parsen

```
parse :: String → ([Int], [Int])
parse = unzip . map (toPair . map read . words) . lines
  where toPair [x, y] = (x, y)
```

Lees van rechts naar links:

```
"3   4\n1   7\n5   2"
    -- lines
["3   4", "1   7", "5   2"]
    -- map (toPair . map read . words)
[(3,4), (1,7), (5,2)]
    -- unzip
([3,1,5], [4,7,2])
```

Oplossing 1: afstanden optellen

```haskell
import Data.List (sort)

distance :: Int → Int → Int
distance x y = abs (x - y)

totalDistance :: [Int] → [Int] → Int
totalDistance xs ys =
  sum (zipWith distance (sort xs) (sort ys))

solve :: String → Int
solve = uncurry totalDistance . parse
```

Oplossing 1: afstanden optellen

```haskell
import Data.List (sort)

distance :: Int → Int → Int
distance x y = abs (x - y)

totalDistance :: [Int] → [Int] → Int
totalDistance xs ys =
  sum (zipWith distance (sort xs) (sort ys))

solve :: String → Int
solve = uncurry totalDistance . parse
```

```haskell
totalDistance [3,1,5] [4,7,2]
  = sum (zipWith distance [1,3,5] [2,4,7])
  = sum [1, 1, 2]  =  4
```

Opdracht 1, deel 2: similarity score

Neem elk getal x uit de linkerkolom:

- tel hoe vaak x in de rechterkolom staat
- vermenigvuldig x met dat aantal

Tel alles op.

Invoer:

```
1 2
2 5
2 2
5 8
```

Opdracht 1, deel 2: similarity score

Neem elk getal x uit de linkerkolom:

- tel hoe vaak x in de rechterkolom staat
- vermenigvuldig x met dat aantal

Tel alles op.

Invoer:

```
1 2
2 5
2 2
5 8
```

$1 \cdot 0 + 2 \cdot 2 + 2 \cdot 2 + 5 \cdot 1$

Uitvoer: **13**

Oplossing 1, deel 2

```haskell
count :: Int → [Int] → Int
count x = length . filter (== x)

similarity :: [Int] → [Int] → Int
similarity xs ys = sum (map score xs)
  where score x = x * count x ys

solveTwo :: String → Int
solveTwo = uncurry similarity . parse
```

Oplossing 1, deel 2

```haskell
count :: Int → [Int] → Int
count x = length . filter (== x)

similarity :: [Int] → [Int] → Int
similarity xs ys = sum (map score xs)
  where score x = x * count x ys

solveTwo :: String → Int
solveTwo = uncurry similarity . parse
```

`parse` uit deel 1 kan je ongewijzigd hergebruiken

Mirage Maintenance (AoC 2023, dag 9)

Elke regel is een reeks metingen. Voorspel de volgende waarde:

1. neem de verschillen tussen buren, tot een rij helemaal `0` is
2. vul van onder naar boven elke rij aan met één getal

```
1   4   9  16  25
  3   5   7   9
    2   2   2
      0   0
```

Invoer:

```
1 4 9 16 25
2 4 6 8
3 3 3
5 1 -1 -1 1
```

Uitvoer (som van de voorspellingen): **54**

`adventofcode.com/2023/day/9`

Mirage Maintenance (AoC 2023, dag 9)

Elke regel is een reeks metingen. Voorspel de volgende waarde:

1. neem de verschillen tussen buren, tot een rij helemaal `0` is
2. vul van onder naar boven elke rij aan met één getal

```
1   4   9  16  25  36
  3   5   7   9  11
    2   2   2   2
      0   0   0
```

Invoer:

```
1 4 9 16 25
2 4 6 8
3 3 3
5 1 -1 -1 1
```

Uitvoer (som van de voorspellingen): **54**

`adventofcode.com/2023/day/9`

Oplossing 2

Begin bovenaan: wat moet er met de hele invoer gebeuren?

```
solve :: String → Int
solve = sum . map next . parse

parse :: String → [[Int]]
parse = map (map read . words) . lines

differences :: [Int] → [Int]
differences xs = undefined

pyramid :: [Int] → [[Int]]
pyramid = undefined

next :: [Int] → Int
next = undefined
```

Oplossing 2

Begin bovenaan: wat moet er met de hele invoer gebeuren?

```
solve :: String → Int
solve = sum . map next . parse

parse :: String → [[Int]]
parse = map (map read . words) . lines

differences :: [Int] → [Int]
differences xs = undefined

pyramid :: [Int] → [[Int]]
pyramid = undefined

next :: [Int] → Int
next = undefined
```

Oplossing 2: de piramide

```
differences :: [Int] → [Int]
differences xs = zipWith (-) (tail xs) xs
-- [4,9,16,25] - [1,4,9,16,25]  →  [3,5,7,9]
```

Oplossing 2: de piramide

```haskell
differences :: [Int] → [Int]
differences xs = zipWith (-) (tail xs) xs
-- [4,9,16,25] - [1,4,9,16,25]  →  [3,5,7,9]
```

```haskell
pyramid :: [Int] → [[Int]]
pyramid = takeWhile (any (/= 0)) . iterate differences
```

```haskell
iterate differences [1,4,9,16,25]
  = [[1,4,9,16,25], [3,5,7,9], [2,2,2], [0,0], ...]
pyramid [1,4,9,16,25]
  = [[1,4,9,16,25], [3,5,7,9], [2,2,2]]
```

`iterate` maakt een oneindige lijst, maar door laziness berekent `takeWhile` enkel de rijen die nodig zijn.

Oplossing 2: voorspellen

De nieuwe waarde is de som van de laatste elementen van elke rij:

```
1   4   9  16  25  |  36 = 25 + 11
  3   5   7   9    |  11 =  9 +  2
    2   2   2      |   2 =  2 +  0
```

Oplossing 2: voorspellen

De nieuwe waarde is de som van de laatste elementen van elke rij:

```
1   4   9  16  25  |  36 = 25 + 11
  3   5   7   9    |  11 =  9 +  2
    2   2   2      |   2 =  2 +  0
```

```haskell
next :: [Int] → Int
next = sum . map last . pyramid
-- map last (pyramid [1,4,9,16,25])  =  [25, 9, 2]
-- sum [25, 9, 2]                    =  36
```

Opdracht 2, deel 2: terug in de tijd

Voorspel nu de vorige waarde: vul elke rij aan de linkerkant aan, weer van onder naar boven.

```
0   1   4   9  16  25  |  0 = 1 - 1
  1   3   5   7   9    |  1 = 3 - 2
    2   2   2   2      |  2 = 2 - 0
      0   0   0
```

Opdracht 2, deel 2: terug in de tijd

Voorspel nu de vorige waarde: vul elke rij aan de linkerkant aan, weer van onder naar boven.

```
0   1   4   9  16  25  |  0 = 1 - 1
  1   3   5   7   9    |  1 = 3 - 2
    2   2   2   2      |  2 = 2 - 0
      0   0   0
```

Het nieuwe getal x moet zo gekozen zijn dat het verschil met zijn rechterbuur klopt:

$$\text{eerste} - x = \text{nieuw getal eronder} \quad \Rightarrow \quad x = \text{eerste} - \text{nieuw getal eronder}$$

Opdracht 2, deel 2: terug in de tijd

Voorspel nu de vorige waarde: vul elke rij aan de linkerkant aan, weer van onder naar boven.

```
0   1   4   9  16  25  |  0 = 1 - 1
  1   3   5   7   9    |  1 = 3 - 2
    2   2   2   2      |  2 = 2 - 0
      0   0   0
```

Het nieuwe getal x moet zo gekozen zijn dat het verschil met zijn rechterbuur klopt:

$$\text{eerste} - x = \text{nieuw getal eronder} \quad \Rightarrow \quad x = \text{eerste} - \text{nieuw getal eronder}$$

Voor de hele invoer: $0 + 0 + 3 + 11 = \mathbf{14}$

Oplossing 2, deel 2

Idee 1: de vorige waarde is de volgende waarde van de omgekeerde lijst.
Hergebruik `next`!

```
solveTwo :: String → Int
solveTwo = sum . map (next . reverse) . parse
-- next [25,16,9,4,1]  =  1 + (-3) + 2  =  0
```

Oplossing 2, deel 2

Idee 1: de vorige waarde is de volgende waarde van de omgekeerde lijst. Hergebruik `next`!

```
solveTwo :: String → Int
solveTwo = sum . map (next . reverse) . parse
-- next [25,16,9,4,1]  =  1 + (-3) + 2  =  0
```

Idee 2: eerste element min de nieuwe waarde eronder, van onder naar boven.

```
previous :: [Int] → Int
previous = foldr ((-) . head) 0 . pyramid
-- map head (pyramid [1,4,9,16,25])  =  [1, 3, 2]
-- foldr (-) 0 [1, 3, 2]  =  1 - (3 - (2 - 0))  =  0
```

Aint no way dat we optijd bij deze slide komen
Bedankt voor de aandacht!